

02_rules-py

#python

#api

#fastapi

#swagger

#pydantic

#PostgreSQL

#SQLAlchemy

#backend

#SIEM

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ api/  
            └─ routes/  
                └─ rules.py
```

El archivo `rules.py` se encuentra dentro de la carpeta de rutas de la API:

```
backend/app/api/routes/
```

Este archivo define los endpoints relacionados con la gestión de reglas de detección del laboratorio SIEM MVP.

Las rutas principales son:

```
POST /rules  
GET /rules
```

Su función es permitir crear reglas nuevas y consultar las reglas existentes almacenadas en PostgreSQL.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,360p' backend/app/api/routes/rules.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,360p'
```

Indica que se impriman las líneas de la 1 a la 360.

```
backend/app/api/routes/rules.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
# backend/app/api/routes/rules.py  
  
from __future__ import annotations
```

```

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.models.rule import Rule
from app.schemas.rule import RuleCreate, RuleOut

router = APIRouter(prefix="/rules", tags=["rules"])

@router.post("", response_model=RuleOut)
def create_rule(payload: RuleCreate, db: Session = Depends(get_db)):
    rule = Rule(
        name=payload.name,
        enabled=payload.enabled,
        source=payload.source,
        severity_min=payload.severity_min,
        contains=payload.contains,
        meta_match=payload.meta_match,
        throttle_seconds=payload.throttle_seconds,
        threshold_count=payload.threshold_count,
        threshold_seconds=payload.threshold_seconds,
    )

    db.add(rule)
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Rule name already exists")

    db.refresh(rule)
    return rule

@router.get("", response_model=list[RuleOut])
def list_rules(
    limit: int = Query(100, ge=1, le=500),
    db: Session = Depends(get_db),
):
    stmt = select(Rule).order_by(Rule.id.desc()).limit(limit)
    return db.execute(stmt).scalars().all()

```

4 Función general del archivo

El archivo `rules.py` define la API de gestión de reglas.

Permite realizar dos operaciones principales:

```

POST /rules → crear una regla nueva
GET /rules → listar reglas existentes

```

Este archivo no evalúa directamente los eventos. Su función es configurar las reglas que después serán utilizadas por el endpoint de ingesta.

La relación funcional es:

```

POST /rules
↓
crea Rule
↓

```

```
guarda en PostgreSQL
    ↓
POST /ingest
    ↓
consulta reglas activas
    ↓
evalúa eventos
    ↓
genera Alert si corresponde
```

Por tanto, `rules.py` forma parte de la fase de configuración del motor de reglas.

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
from __future__ import annotations
```

Importación futura para anotaciones modernas.

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session
```

Importaciones externas de FastAPI y SQLAlchemy.

```
from app.db.session import get_db
from app.models.rule import Rule
from app.schemas.rule import RuleCreate, RuleOut
```

Importaciones internas del proyecto.

```
router = APIRouter(prefix="/rules", tags=["rules"])
```

Creación del router `/rules`.

```
@router.post("", response_model=RuleOut)
def create_rule(...):
```

Endpoint para crear reglas.

```
@router.get("", response_model=list[RuleOut])
def list_rules(...):
```

Endpoint para listar reglas.

Visualmente:

```
rules.py
├─ Importaciones
├─ Router /rules
├─ POST /rules
│   ├── recibe RuleCreate
│   ├── crea Rule
│   ├── db.add()
│   ├── db.commit()
│   ├── controla IntegrityError
│   ├── db.refresh()
│   └─ devuelve RuleOut
└─ GET /rules
    └─ recibe limit
```

```
| select(Rule)
| order_by(Rule.id.desc())
| limit(limit)
| devuelve list[RuleOut]
```

6 Análisis línea por línea

Comentario inicial

```
# backend/app/api/routes/rules.py
```

Esta línea es un comentario.

En Python, todo lo que empieza por `#` no se ejecuta.

Aquí sirve para indicar la ubicación del archivo dentro del proyecto.

No afecta al funcionamiento de la API.

Importación futura de anotaciones

```
from __future__ import annotations
```

Esta línea activa el comportamiento moderno de Python para anotaciones de tipos.

Permite trabajar de forma más flexible con anotaciones y referencias de tipos.

En este archivo no hay una anotación compleja que dependa directamente de esto, pero mantiene coherencia con otros archivos del proyecto.

Importación de FastAPI

```
from fastapi import APIRouter, Depends, HTTPException, Query
```

Esta línea importa cuatro elementos de FastAPI:

```
APIRouter
Depends
HTTPException
Query
```

APIRouter

`APIRouter` permite definir rutas en un archivo separado de `main.py`.

En este archivo se usa aquí:

```
router = APIRouter(prefix="/rules", tags=["rules"])
```

Después, este router se registra en la aplicación principal desde `main.py`:

```
app.include_router(rules_router)
```

Depends

`Depends` permite usar dependencias de FastAPI.

En este archivo se utiliza para obtener una sesión de base de datos:

```
db: Session = Depends(get_db)
```

Esto significa que FastAPI ejecuta `get_db()` y entrega una sesión SQLAlchemy al endpoint.

HTTPException

`HTTPException` permite devolver errores HTTP controlados.

En este archivo se usa cuando se intenta crear una regla con un nombre ya existente:

```
raise HTTPException(status_code=409, detail="Rule name already exists")
```

Query

`Query` permite definir parámetros de consulta con validaciones.

En este archivo se usa en el endpoint `GET /rules`:

```
limit: int = Query(100, ge=1, le=500)
```

Esto controla el número máximo de reglas devueltas.

Importación de `select`

```
from sqlalchemy import select
```

Esta línea importa `select` desde SQLAlchemy.

`select` permite construir consultas SQL usando sintaxis Python.

En este archivo se usa aquí:

```
stmt = select(Rule).order_by(Rule.id.desc()).limit(limit)
```

Conceptualmente equivale a una consulta SQL como:

```
SELECT * FROM rules ORDER BY id DESC LIMIT 100;
```

Importación de `IntegrityError`

```
from sqlalchemy.exc import IntegrityError
```

Esta línea importa la excepción `IntegrityError` desde SQLAlchemy.

`IntegrityError` se produce cuando la base de datos rechaza una operación por incumplir una restricción.

En este proyecto, el caso principal es intentar crear una regla con un nombre repetido.

El modelo `Rule` tiene el campo `name` marcado como único:

```
name: Mapped[str] = mapped_column(String(120), nullable=False, unique=True)
```

Si se intenta insertar otra regla con el mismo nombre, PostgreSQL lanza un error de integridad y SQLAlchemy lo representa como `IntegrityError`.

Importación de `Session`

```
from sqlalchemy.orm import Session
```

Importa el tipo `Session` desde SQLAlchemy ORM.

Se usa como anotación de tipo:

```
db: Session = Depends(get_db)
```

Esto indica que `db` será una sesión de base de datos.

Importación de `get_db`

```
from app.db.session import get_db
```

Importa la función `get_db` desde:

```
backend/app/db/session.py
```

`get_db` proporciona una sesión de base de datos para cada petición.

La relación es:

```
get_db()
  ↓
SessionLocal()
  ↓
db
  ↓
endpoint usa PostgreSQL
  ↓
db.close()
```

Importación del modelo `Rule`

```
from app.models.rule import Rule
```

Importa el modelo SQLAlchemy `Rule`.

Este modelo representa la tabla:

```
rules
```

En este archivo se usa para:

- Crear reglas nuevas.
- Consultar reglas existentes.
- Ordenar reglas por id.

Importación de schemas

```
from app.schemas.rule import RuleCreate, RuleOut
```

Importa dos schemas de Pydantic:

```
RuleCreate
RuleOut
```

`RuleCreate`

Se usa como schema de entrada en:

```
def create_rule(payload: RuleCreate, db: Session = Depends(get_db)):
```

Valida el JSON recibido al crear una regla.

RuleOut

Se usa como schema de salida en:

```
@router.post("", response_model=RuleOut)
```

y:

```
@router.get("", response_model=list[RuleOut])
```

Define cómo se devuelven las reglas desde la API.

Creación del router

```
router = APIRouter(prefix="/rules", tags=["rules"])
```

Esta línea crea el router de reglas.

Desglose:

```
router
```

Variable donde se guarda el router.

```
APIRouter(...)
```

Crea una instancia de router de FastAPI.

```
prefix="/rules"
```

Todas las rutas definidas en este archivo empiezan por `/rules`.

```
tags=["rules"]
```

Agrupar los endpoints en Swagger bajo la etiqueta `rules`.

Como los decoradores usan cadena vacía:

```
@router.post("")
@router.get("")
```

las rutas finales son:

```
POST /rules
GET /rules
```

Endpoint POST /rules

```
@router.post("", response_model=RuleOut)
```

Este decorador registra un endpoint HTTP de tipo `POST`.

La ruta final es:

```
POST /rules
```

El parámetro:

```
response_model=RuleOut
```

indica que la respuesta debe adaptarse al schema `RuleOut`.

Esto significa que, aunque la función devuelva un objeto SQLAlchemy `Rule`, FastAPI lo convierte en JSON con los campos definidos en `RuleOut`.

Definición de `create_rule`

```
def create_rule(payload: RuleCreate, db: Session = Depends(get_db)):
```

Define la función que se ejecuta cuando llega una petición `POST /rules`.

Parámetros:

```
payload → datos validados con RuleCreate
db       → sesión SQLAlchemy proporcionada por get_db
```

FastAPI realiza automáticamente:

1. Lee el JSON recibido.
2. Lo valida usando `RuleCreate`.
3. Ejecuta `get_db()`.
4. Entrega `payload` y `db` a `create_rule()`.

Creación del objeto `Rule`

```
rule = Rule(
    name=payload.name,
    enabled=payload.enabled,
    source=payload.source,
    severity_min=payload.severity_min,
    contains=payload.contains,
    meta_match=payload.meta_match,
    throttle_seconds=payload.throttle_seconds,
    threshold_count=payload.threshold_count,
    threshold_seconds=payload.threshold_seconds,
)
```

Este bloque crea un objeto SQLAlchemy de tipo `Rule`.

Todavía no está guardado definitivamente en PostgreSQL. Primero se crea en memoria como objeto Python.

La relación entre el schema de entrada y el modelo ORM es:

<code>payload.name</code>	→ <code>rule.name</code>
<code>payload.enabled</code>	→ <code>rule.enabled</code>
<code>payload.source</code>	→ <code>rule.source</code>
<code>payload.severity_min</code>	→ <code>rule.severity_min</code>
<code>payload.contains</code>	→ <code>rule.contains</code>
<code>payload.meta_match</code>	→ <code>rule.meta_match</code>
<code>payload.throttle_seconds</code>	→ <code>rule.throttle_seconds</code>
<code>payload.threshold_count</code>	→ <code>rule.threshold_count</code>
<code>payload.threshold_seconds</code>	→ <code>rule.threshold_seconds</code>

Este paso transforma los datos validados por Pydantic en un modelo persistente por SQLAlchemy.

Campo `name`

```
name=payload.name,
```

Asigna el nombre de la regla.

Este campo es obligatorio y único.

Ejemplo:

```
High severity events
Auth failed login
Five auth failures in one minute
```

Campo `enabled`

```
enabled=payload.enabled,
```

Asigna si la regla estará activa o no.

Si `enabled` es `True`, la regla podrá ser evaluada en `/ingest`.

Si `enabled` es `False`, la regla quedará almacenada pero no se usará para detectar eventos.

Campo `source`

```
source=payload.source,
```

Asigna un filtro opcional por origen del evento.

Ejemplo:

```
auth
firewall
linux
```

Si es `None`, la regla no filtrará por origen.

Campo `severity_min`

```
severity_min=payload.severity_min,
```

Asigna la severidad mínima requerida.

Ejemplo:

```
severity_min = 5
```

La regla solo coincidirá con eventos cuya severidad sea igual o superior a 5.

Campo `contains`

```
contains=payload.contains,
```

Asigna el texto que debe aparecer en el mensaje del evento.

Ejemplo:

```
failed login
blocked
denied
```

Si es `None` , la regla no filtra por contenido textual.

Campo `meta_match`

```
meta_match=payload.meta_match,
```

Asigna condiciones exactas sobre el campo `meta` del evento.

Ejemplo:

```
{
  "user": "admin",
  "host": "server-01"
}
```

Durante la ingesta, el evento deberá contener esos pares clave-valor dentro de `meta` .

Campo `throttle_seconds`

```
throttle_seconds=payload.throttle_seconds,
```

Asigna el throttle de la regla en segundos.

Ejemplo:

```
throttle_seconds = 300
```

Esto limita la frecuencia con la que esa regla puede generar alertas para el mismo grupo.

Campo `threshold_count`

```
threshold_count=payload.threshold_count,
```

Asigna el número de eventos necesarios para activar una regla de threshold.

Ejemplo:

```
threshold_count = 5
```

Campo `threshold_seconds`

```
threshold_seconds=payload.threshold_seconds,
```

Asigna la ventana temporal del threshold.

Ejemplo:

```
threshold_seconds = 60
```

Combinado con `threshold_count` , permite expresar:

```
5 eventos en 60 segundos
```

Añadir regla a la sesión

```
db.add(rule)
```

Añade el objeto `Rule` a la sesión SQLAlchemy.

Esto marca el objeto como pendiente de inserción en PostgreSQL.

Todavía no se guarda definitivamente hasta ejecutar:

```
db.commit()
```

Inicio del bloque `try`

```
try:
```

Inicia un bloque de control de errores.

Aquí se intenta confirmar la transacción.

El motivo principal es capturar posibles errores de integridad, especialmente nombres duplicados.

Confirmar la regla en base de datos

```
db.commit()
```

Confirma la transacción.

En este momento, SQLAlchemy intenta insertar la regla en PostgreSQL.

Si todo va bien, la regla queda guardada.

Si hay un problema, como un nombre duplicado, se lanza una excepción.

Captura de `IntegrityError`

```
except IntegrityError:
```

Captura errores de integridad de base de datos.

El caso esperado en este endpoint es que el usuario intente crear una regla con un nombre ya existente.

Esto ocurre porque `name` tiene restricción de unicidad.

Rollback

```
db.rollback()
```

Revierte la transacción fallida.

Cuando una operación de base de datos falla, la sesión queda en estado inválido hasta que se hace rollback.

Este rollback limpia la transacción para evitar problemas posteriores.

Error HTTP 409

```
raise HTTPException(status_code=409, detail="Rule name already exists")
```

Devuelve un error controlado al cliente.

Desglose:

```
status_code=409
```

El código HTTP 409 significa conflicto.

Tiene sentido porque el cliente intenta crear un recurso que entra en conflicto con uno ya existente.

```
detail="Rule name already exists"
```

Mensaje que explica el problema.

Ejemplo de respuesta:

```
{
  "detail": "Rule name already exists"
}
```

Refrescar la regla

```
db.refresh(rule)
```

Actualiza el objeto `rule` con los valores generados por la base de datos.

Esto es importante porque PostgreSQL puede haber generado campos automáticamente, como:

```
id
created_at
```

Después de `db.refresh(rule)`, el objeto tiene esos valores cargados y puede devolverse correctamente como `RuleOut`.

Devolver la regla

```
return rule
```

Devuelve el objeto SQLAlchemy `Rule`.

Como el endpoint tiene:

```
response_model=RuleOut
```

FastAPI convierte el objeto ORM en una respuesta JSON usando el schema `RuleOut`.

Endpoint GET /rules

```
@router.get("", response_model=list[RuleOut])
```

Este decorador registra un endpoint HTTP de tipo `GET`.

La ruta final es:

```
GET /rules
```

El parámetro:

```
response_model=list[RuleOut]
```

indica que la respuesta será una lista de reglas.

Cada elemento de la lista se serializa usando `RuleOut`.

Definición de `list_rules`

```
def list_rules(  
    limit: int = Query(100, ge=1, le=500),  
    db: Session = Depends(get_db),  
):
```

Define la función que lista reglas.

Recibe dos parámetros:

```
limit → número máximo de reglas a devolver  
db    → sesión SQLAlchemy
```

Parámetro `limit`

```
limit: int = Query(100, ge=1, le=500),
```

Define cuántas reglas se devuelven como máximo.

Desglose:

```
limit
```

Nombre del parámetro.

```
: int
```

Debe ser un entero.

```
Query(100, ge=1, le=500)
```

Define valor por defecto y restricciones.

```
100    → valor por defecto  
ge=1   → mínimo 1  
le=500 → máximo 500
```

Ejemplos:

```
GET /rules  
GET /rules?limit=10  
GET /rules?limit=500
```

Si se envía un valor fuera de rango, FastAPI devolverá error de validación.

Parámetro `db`

```
db: Session = Depends(get_db),
```

Obtiene una sesión de base de datos mediante la dependencia `get_db`.

Esto permite consultar la tabla `rules`.

Consulta de reglas

```
stmt = select(Rule).order_by(Rule.id.desc()).limit(limit)
```

Crea la consulta para listar reglas.

Desglose:

```
select(Rule)
```

Selecciona objetos `Rule`.

```
.order_by(Rule.id.desc())
```

Ordena por `id` descendente.

Esto hace que las reglas más recientes aparezcan primero.

```
.limit(limit)
```

Limita el número de resultados.

Conceptualmente equivale a:

```
SELECT *
FROM rules
ORDER BY id DESC
LIMIT 100;
```

Ejecución de la consulta

```
return db.execute(stmt).scalars().all()
```

Ejecuta la consulta y devuelve los resultados.

Desglose:

```
db.execute(stmt)
```

Ejecuta la consulta SQLAlchemy.

```
.scalars()
```

Extrae los objetos `Rule` directamente.

```
.all()
```

Devuelve todos los resultados como una lista.

Como el endpoint tiene:

```
response_model=list[RuleOut]
```

FastAPI convierte esa lista de objetos ORM en una lista JSON.

Resultado final del archivo

Este archivo expone dos endpoints:

```
POST /rules
GET /rules
```

POST /rules :

1. Recibe un RuleCreate.
2. Crea un objeto Rule.
3. Lo añade a la sesión.
4. Intenta confirmar la transacción.
5. Si el nombre está duplicado, devuelve 409.
6. Refresca el objeto.
7. Devuelve RuleOut.

GET /rules :

1. Recibe un parámetro limit opcional.
2. Construye select(Rule).
3. Ordena por id descendente.
4. Limita resultados.
5. Devuelve list[RuleOut].

7 Relación con el flujo técnico del laboratorio

`rules.py` forma parte de la fase de configuración del motor de reglas.

Primero se crean reglas:

```
POST /rules
  ↓
RuleCreate
  ↓
Rule
  ↓
tabla rules
```

Después, durante la ingesta, esas reglas se utilizan:

```
POST /ingest
  ↓
se crea Event
  ↓
se consultan Rule activas
  ↓
se evalúan condiciones
  ↓
si coincide, se crea Alert
```

Por tanto, `rules.py` no genera alertas directamente, pero permite definir las condiciones que después se usarán para generarlas.

8 Errores típicos o puntos importantes

Nombre de regla duplicado

Si se crea una regla con un nombre ya existente, PostgreSQL genera un error de integridad.

El código lo captura con:

```
except IntegrityError:
```

y devuelve:

```
409 Conflict
```

con el mensaje:

```
Rule name already exists
```

`db.rollback()` es obligatorio tras error de commit

Después de un `IntegrityError`, la sesión queda en estado fallido.

Por eso se ejecuta:

```
db.rollback()
```

Sin rollback, la sesión podría no poder reutilizarse correctamente.

`db.refresh(rule)` carga valores generados por la base de datos

Después del `commit`, la base de datos puede haber generado:

```
id
created_at
```

`db.refresh(rule)` actualiza el objeto Python con esos valores.

`GET /rules` no filtra por `enabled`

El endpoint lista reglas sin distinguir si están activas o inactivas.

Esto significa que devuelve tanto:

```
enabled = true
enabled = false
```

La evaluación de reglas activas se realiza en `ingest.py`, no aquí.

`limit` evita respuestas demasiado grandes

El parámetro `limit` está limitado entre 1 y 500.

Esto evita que el endpoint devuelva demasiadas reglas de golpe.

`POST /rules` no valida coherencia entre `threshold_count` y `threshold_seconds`

El schema permite que uno de los dos sea `None`.

La lógica de `ingest.py` solo aplica `threshold` si ambos existen:

```
if rule.threshold_count is not None and rule.threshold_seconds is not None:
```

Por tanto, una regla con solo uno de esos campos no aplicará `threshold` real.

9 Comandos útiles relacionados

Crear regla simple por severidad:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "High severity events",
  "enabled": true,
```



```
    "severity_min": 5
  }'
```

Crear regla por origen:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Auth events",
  "enabled": true,
  "source": "auth"
}'
```

Crear regla por texto:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Failed login messages",
  "enabled": true,
  "contains": "failed login"
}'
```

Crear regla con throttle:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Throttle failed logins",
  "enabled": true,
  "source": "auth",
  "contains": "failed login",
  "throttle_seconds": 300
}'
```

Crear regla con threshold:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Five failed logins in one minute",
  "enabled": true,
  "source": "auth",
  "contains": "failed login",
  "threshold_count": 5,
  "threshold_seconds": 60
}'
```

Crear regla con meta_match:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Admin activity",
  "enabled": true,
  "meta_match": {
    "user": "admin"
  }
}'
```

Listar reglas:

```
curl http://localhost:8000/rules
```

Listar 10 reglas:

```
curl "http://localhost:8000/rules?limit=10"
```

Probar error por nombre duplicado:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "High severity events",
  "enabled": true,
  "severity_min": 5
}'
```

Consultar reglas directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, enabled, source, severity_min,
contains, throttle_seconds, threshold_count, threshold_seconds, meta_match, created_at FROM rules ORDER
BY id DESC LIMIT 10;"
```

Comprobar reglas activas:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name FROM rules WHERE enabled IS TRUE;"
```

Probar importación del router:

```
docker exec -it siem-api python -c "from app.api.routes.rules import router; print(router)"
```

Ver Swagger:

```
http://localhost:8000/docs
```